Cmpsc 448 Midterm Project

Team: Sungmin Song (individual project)

1. Problem definition

I study LLM authorship attribution as a text classification problem. Each example is a triple (LLM_name, LLM_input, LLM_output). The goal is to predict LLM_name (GPT, Claude, Gemini or Llama) from the text. All four models are fluent and answer the same prompts, so a classifier cannot rely on topic or correctness. It has to learn each model's habits in structure, formatting and word choice. I study four questions:

1. **RQ1.** Can the LLM be identified from its output alone?
2. **RQ2.** Does the user's prompt help (input only vs output only vs input + output)?
3. **RQ3.** Does a classifier trained without a task type still work on that task type?
4. **RQ4.** Which characteristics (formatting, punctuation, vocabulary, length) distinguish the models?

Reliable attribution matters for detecting undisclosed AI-generated text and for understanding how strongly training gives each model a recognizable style.

2. Dataset curation

The dataset has 3,216 responses: 804 identical instructions, each answered by all four models. It is built from public data, so it contains no private or personal information.

Source. AlpacaEval (Li et al., 2023) publishes each evaluated model's answers to the same fixed set of 805 instructions. The instructions come from five public collections (self-instruct, OpenAssistant, Koala, Anthropic helpful-base, Vicuna). The data permits non-commercial use with attribution. `get_public_data.py` downloads and processes it.

Label	Family	Model (AlpacaEval name)
gpt	GPT (OpenAI)	gpt-4o-2024-05-13
claude	Claude (Anthropic)	claude-3-5-sonnet-20240620
llama	Llama (Meta)	Meta-Llama-3.1-70B-Instruct-Turbo
gemini	Gemini (Google)	gemini-pro

- One instruction had an empty Gemini answer. It was dropped for all four models, leaving 804 prompts and exactly 804 examples per class.

Task labels for RQ3. AlpacaEval has no task labels, so I assigned them with keyword rules:

Task group	Rule (first match wins)	Prompts

Task group	Rule (first match wins)	Prompts
technical	coding words (code, python, SQL, function, …) or explicit math words (calculate, equation, probability, …)	84
writing	write, poem, story, essay, email, rewrite, …	128
general_qa	everything else	592

I checked the rules by hand. I removed two early rules because they mislabeled often: digit patterns ("70-75 dB" and "9-11 am" were tagged as math) and the phrases "how many" and "solve" (which caught trivia and crossword questions). The labels remain heuristic.

Limitations.

- The generation settings (temperature, system prompt) were chosen by AlpacaEval contributors.

3. Models and training

Both required models share a front end and a classifier head. They differ only in how a variable-length text becomes one fixed-size vector. Both are trained from scratch with no pretrained embeddings.

Evaluation protocol.

- **Split by prompt** into 70/15/15, stratified by task: 562 training prompts (2,248 responses), 121 validation and 121 test prompts. All four responses to a prompt land in the same split, so test prompts are never seen in training.
- **Seeds.** Each number is the mean ± standard deviation over 3 seeds. Each seed changes both the split and the initialization.
- **Metrics:** accuracy and macro-F1.

Shared front end.

- **Tokenizer:** a regex that keeps words, punctuation marks and newlines as separate tokens, with case preserved. Formatting may be part of the fingerprint, so it is not normalized away.
- **Vocabulary:** built from the training split only (tokens with frequency ≥ 2), about 20,000 tokens.
- **Truncation:** outputs at 600 tokens, inputs at 64.
- **Embedding:** each token maps to a learned 128-dimensional vector. A response becomes a matrix E of size T × 128.

CNN (after Kim, 2014).

- Parallel 1-D convolutions with filter widths 3, 4 and 5, 128 filters each, followed by ReLU. Each filter acts as a learned n-gram detector.
- Masked max-over-time pooling keeps each filter's strongest response anywhere in the text: "did this pattern occur?". Padding positions are masked so they can never be the maximum.

- The 384 pooled values pass through dropout (0.5) and a linear layer to 4 classes. The model has 2.76M parameters, 2.56M of them in the embedding table.

$$c_i = \mathrm{ReLU}(w \cdot [e_i; \ldots; e_{i+h-1}] + b), \qquad \hat{c} = \max_i c_i$$

BiLSTM (RNN).

- A bidirectional LSTM with 128 hidden units per direction, run over packed (unpadded) sequences.
- The text vector concatenates the final hidden states of both directions (256) with a max-pool over all time steps (256), giving 512 values. The max-pool (as in Conneau et al., 2017) helps on long responses, where the final state mostly reflects the end of the text.
- Dropout is 0.5 on the embeddings and before the linear layer. The model has 2.83M parameters.

Training (both models).

- **Loss and optimizer:** cross-entropy, Adam (learning rate 10^{-3}), batch size 32, gradient clipping at norm 5.
- **Early stopping:** up to 15 epochs, stopping when validation macro-F1 has not improved for 4 epochs; the best epoch's weights are restored. The test set is touched only once.
- **Hardware:** an NVIDIA T4 GPU on Google Colab. The full experiment suite (about 90 trained models) took 43 minutes.

$$p(y = k \mid x) = \mathrm{softmax}(Wz + b)_k, \qquad \mathcal{L} = -\log p(y_{\text{true}} \mid x)$$

Baseline (TF-IDF + logistic regression). As a reference point for the two required models, I also trained a simple non-neural classifier. Each response becomes a vector of TF-IDF scores over word 1–2-grams and character 2–4-grams: how often each short word sequence and character sequence appears, down-weighted if it is common across all responses. An L2-regularized logistic regression (C = 10) then predicts the LLM from that vector. The model only counts surface patterns, with no learned word meanings and no word order beyond 2 words. So if the CNN or BiLSTM cannot beat it, they are not learning anything beyond n-gram counting.

4. Results

RQ1: identifying the LLM from its output

Yes. From output text alone, the CNN reaches 0.815 accuracy and the BiLSTM 0.771, about three times chance (0.25).

Model	Accuracy	Macro-F1
CNN	0.815 ± 0.024	0.816 ± 0.025
BiLSTM	0.771 ± 0.004	0.768 ± 0.004
TF-IDF + LogReg (baseline)	0.824 ± 0.011	0.822 ± 0.012

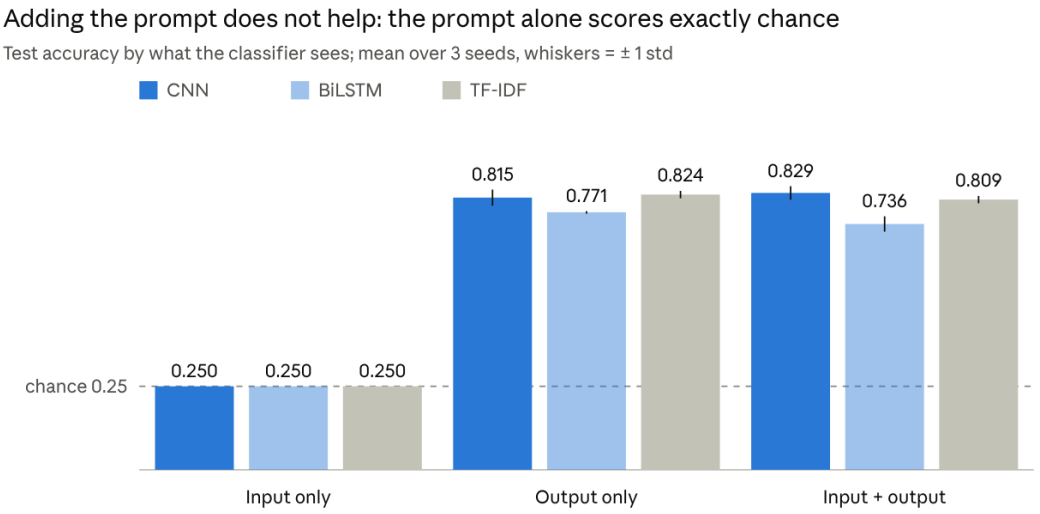
- **CNN vs baseline.** The CNN and the TF-IDF baseline are statistically tied (the difference is smaller than one standard deviation). Both mainly detect short telltale n-grams: the CNN learns its n-gram detectors, while TF-IDF counts fixed ones.
- **BiLSTM.** It is about 4 points lower but the most stable across seeds. RQ4 shows its gap to the CNN shrinks when inputs are cut to 50 tokens, which points to long sequences as its difficulty.
- **Per model (CNN).** Recall is 0.85 for Claude, 0.83 for Gemini, 0.82 for Llama and 0.75 for GPT. The most common error is GPT predicted as Gemini (18% of GPT responses).
- **Per task.** Writing prompts are the hardest for every model:

Model	general_qa	technical	writing
CNN	0.841	0.763	0.732
BiLSTM	0.791	0.821	0.645
TF-IDF + LogReg	0.838	0.814	0.763

Only about 13 technical and 19 writing prompts fall in each seed's test split, so the per-task numbers are noisy.

RQ2: does the user's prompt help?

No. The prompt alone gives exactly chance accuracy, and adding it to the output does not improve accuracy.



- **(i) Can the LLM be predicted from the prompt?** Not in this dataset. Every prompt was answered by all four models, so the prompt carries no information about the label. Each model learns to predict one class for every example, which gives exactly 0.25 accuracy. Macro-F1 falls below 0.25 (0.10–0.22) because three of the four classes are never predicted.
- **(ii) Does adding the input help?** No. The CNN's gain (0.815 → 0.829) is within noise. The BiLSTM gets worse (0.771 → 0.736), because the prompt tokens lengthen the sequence without adding signal.

- **(iii) Why could input-only work "surprisingly well" elsewhere?** In scraped real-world data, different users, topics, languages and time periods reach different models (for example, coding questions sent mostly to one assistant). The prompt then predicts the model through *who asked and when*, not through *who answered*. That is dataset leakage, which my design removes by construction. A high input-only score on another dataset should be read as a warning sign, not as a fingerprint.

5. More results and in-depth analysis

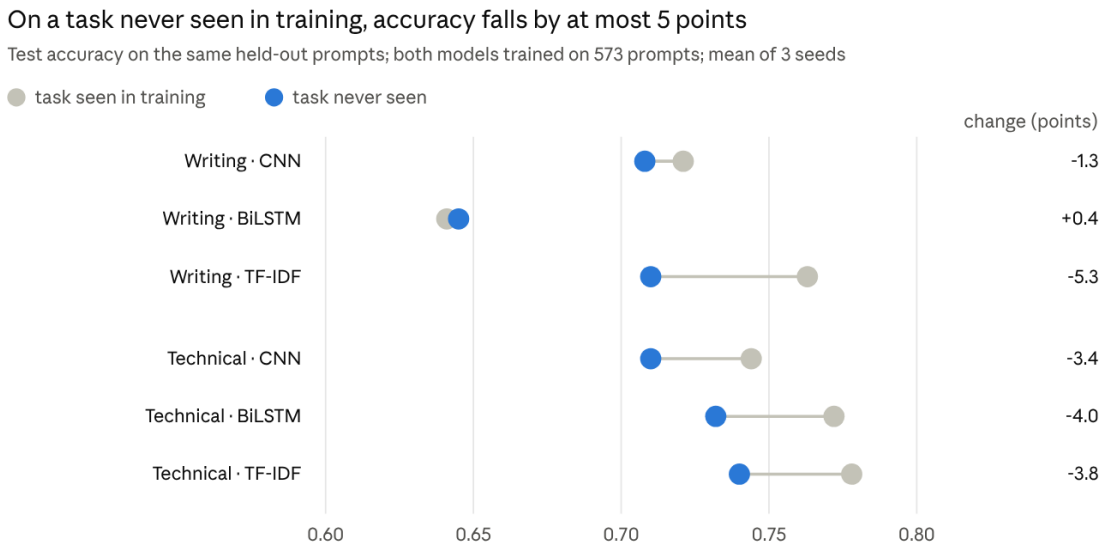
RQ3: do fingerprints carry over to unseen tasks?

Mostly yes. A classifier that never saw a task type loses at most 5.3 points on it, and its accuracy stays above 0.64 for every model.

Design: a paired, size-matched comparison. A naive leave-one-task-out test confounds two effects. Holding out general QA would cut training from 562 to 212 prompts, so any drop would mix domain shift with having less data. I instead compare two classifiers for each held-out group G (writing, technical):

1. G's prompts are split in half: a test half and a pool half.
2. **Unseen:** trained on 573 prompts from the other groups only.
3. **Seen:** trained on G's pool half plus other-group prompts, also 573 in total.
4. Both use the same validation set and are tested on the same test half of G.

The gap therefore measures only the effect of having seen the task. General QA is not held out, because removing it leaves too little training data.



results/rq3.json · writing: 64 test prompts, technical: 42, per seed · std 0.01–0.04

- **(i) Does performance decrease on unseen tasks?** Slightly. The drops are 1.3–3.4 points for the CNN, −0.4 to 4.0 for the BiLSTM and 3.8–5.3 for TF-IDF. With standard deviations of 0.01–0.04 and only 42–64 test prompts, the CNN and BiLSTM drops on writing are within noise.
- **(ii) Are some models more consistently identifiable?** Yes. Claude stays recognizable on unseen tasks, while GPT is the least robust, especially on writing:

Held-out task	Classifier	Claude	Gemini	GPT	Llama
writing	CNN	0.77 → 0.86	0.84 → 0.74	0.56 → 0.53	0.71 → 0.70
writing	BiLSTM	0.89 → 0.88	0.72 → 0.71	0.37 → 0.40	0.58 → 0.59
technical	CNN	0.89 → 0.89	0.70 → 0.64	0.76 → 0.74	0.63 → 0.57
technical	BiLSTM	0.95 → 0.91	0.79 → 0.79	0.65 → 0.70	0.69 → 0.53

Recall per model, seen → unseen. A plausible reason for Claude's robustness is that its strongest habit (almost never using bold, RQ4) does not depend on the task.

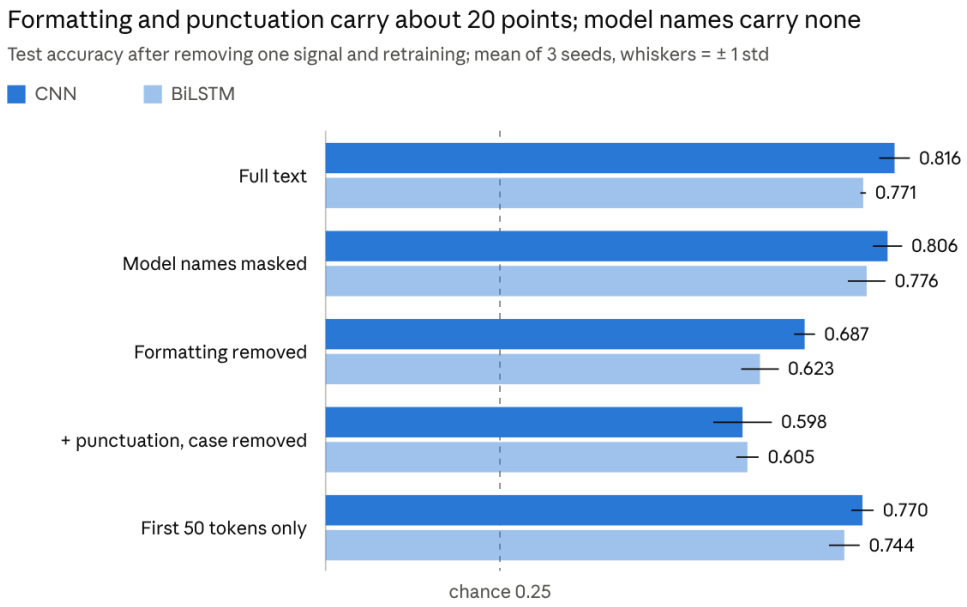
- **(iii) General fingerprint or task-specific patterns?** Mostly general. TF-IDF, which relies most on specific words, drops the most. The CNN and BiLSTM, which also pick up formatting and punctuation patterns, drop less. The task-specific part of the fingerprint is real but small.

RQ4: what distinguishes the models?

Feature (per response)	Claude	Gemini	GPT	Llama
Words	230	226	283	324
Lexical diversity (TTR, first 200 words)	0.64	0.62	0.63	0.59
Words per sentence	17.3	20.9	17.3	21.9
Markdown headers	0.13	0.11	1.66	0.23
Bold spans	0.01	4.24	7.54	7.50
Numbered-list items	7.33	3.58	5.08	5.87
Bullet items	3.56	6.10	6.58	3.94
LaTeX commands	0.00	0.03	0.58	0.02
Exclamation marks	0.44	0.15	0.67	0.77

Each model has distinct habits. Claude almost never uses bold; GPT is the only one using headers and LaTeX; Llama writes the longest answers with the most "!"; Gemini prefers bullets over numbered lists and uses the fewest exclamation marks. The most indicative phrases match this: "Sure!" and "##" for GPT, "Here's" for Claude, "Here is" for Llama, and "It is important" for Gemini. Style counts alone reach 0.573 accuracy, so word choice adds information beyond them.

Removing signals (ablations). Each variant changes the text and retrains both models from scratch:



results/rq4.json · output only · 3 seeds; each variant retrained from scratch

- **(i) What signals does the classifier use?** The classifier does not rely on models naming themselves: masking names changes nothing. It relies on formatting and punctuation and case.
- **(ii) Linguistic, structural, or both?** Both kinds of signal matter. Structure accounts for about 17–22 points, and words alone still reach about 0.60.
- **(iii) Does removing signals substantially reduce performance?** Removing formatting and punctuation hurts substantially, but length barely matters: the first 50 tokens alone lose only 3–5 points. The BiLSTM's gap to the CNN also narrows from 4.5 to 2.6 points on these short inputs, consistent with long sequences being its main difficulty.

6. Lessons and experience

I used Claude to get help with building the structures and analyzing the data. It would've been a lot harder if I hadn't used Claude.

The main lesson of this project is that careful experimental design matters as much as the choice of model. A simple TF-IDF baseline tied CNN and beat BiLSTM. With only 2,248 training responses and

word embeddings learned from scratch, the neural models had little advantage over plain word counting, and without the baseline I would have overrated them. I also learned to look for hidden problems in setups that seemed correct. Every response to a prompt had to stay in the same split, so the models were always tested on unseen prompts. The first RQ3 design gave the unseen model far less training data (212 vs 562 prompts), which I fixed by matching training sizes. Checking the data by hand caught a file stored in a different order and keyword rules that tagged "70-75 dB" as math, either of which would have silently corrupted the results. The training curves helped explain the gap between the models: the CNN began overfitting around epoch 10, which early stopping caught, while the BiLSTM was still improving when it reached the 15-epoch limit.

The study has limitations. I did not control how the responses were generated, Gemini Pro is older than the other three models, each family is represented by a single version, and the task labels are heuristic.

Future work could use pretrained embeddings or a transformer encoder, measure a learning curve to see whether more data helps, and add more model families and versions.

7. Code and references

Code and data. GitHub repository: https://github.com/sungmin0911/Cmpsc-448-Midterm-project

It contains:

- `get_public_data.py`: data download and task labels.
- `train.py`: models, training and all four RQs.
- `report_figs.py`: figures and tables.
- `llm_fingerprint.ipynb`: a Colab notebook that runs everything end to end.
- `data/responses.jsonl` and `results/`.

The full run is `python get_public_data.py && python train.py --rq 1 2 3 4 && python report_figs.py`.

References.

- Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P., & Hashimoto, T. B. (2023). *AlpacaEval: An Automatic Evaluator of Instruction-following Models.* github.com/tatsu-lab/alpaca_eval
- Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *EMNLP 2014.*
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation, 9(8).*
- Conneau, A., Kiela, D., Schwenk, H., Barrault, L., & Bordes, A. (2017). Supervised Learning of Universal Sentence Representations from Natural Language Inference Data. *EMNLP 2017.*
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015.*